

Public Dental Support RAG Chatbot — Goal-Based Project Document

1. Project Name

Public Dental Support AI Chatbot

2. Project Type

RAG-Based Public Customer Support Chatbot

RAG means Retrieval-Augmented Generation. The chatbot will not only generate answers from AI knowledge. It will retrieve approved dental support content from a knowledge base and then generate a safe, professional, source-based response.

3. Project Vision

The goal of this project is to build a professional, client-attractive, and public-facing AI chatbot for dental support. The chatbot will help website visitors understand general dental claims, dental codes, claim rejections, claim denials, insurance terms, required documents, and next steps.

The chatbot should provide clear, trustworthy, and user-friendly guidance while maintaining strong privacy boundaries. It will not access private claim records, patient data, subscriber IDs, or live insurance systems.

4. Main Project Goal

To create a high-quality public dental support chatbot that can answer general user questions using a verified dental knowledge base and provide professional guidance related to dental claims, coding, insurance, and claim processing issues.

The chatbot should improve user experience, reduce repetitive support questions, and create a modern AI-powered impression for the client's website.

5. Target Users

The chatbot is designed for public users, including:

- Website visitors
- Dental patients

- Dental customers
 - People asking about dental claim processing
 - People confused about dental codes
 - People trying to understand claim rejection or denial reasons
 - People looking for general insurance-related dental guidance
-

6. What the Chatbot Will Help With

The chatbot will answer general questions about:

- Dental claims
 - Dental procedure codes
 - Claim rejection reasons
 - Claim denial reasons
 - Dental insurance terms
 - Pre-authorization
 - Required claim documents
 - Attachments such as X-rays or supporting documents
 - Tooth number and tooth surface requirements
 - Coordination of benefits
 - Claim resubmission guidance
 - Appeal process guidance
 - General next steps when a claim fails
-

7. What the Chatbot Will Not Do

The chatbot will not:

- Check real claim status
- Ask for patient private information
- Ask for subscriber ID
- Ask for Social Security number
- Ask for date of birth
- Access patient records
- Access insurance accounts
- Access internal claim logs
- Give final clinical advice
- Give final legal or insurance decisions
- Replace the dental office, billing team, or insurance carrier

The chatbot must clearly explain that it provides general guidance only.

8. Public Safety Rule

The chatbot should always follow this rule:

Provide general dental claims and coding guidance only. Do not collect or process private patient or insurance information. For exact claim status or final decisions, guide the user to contact their dental office, billing team, or insurance carrier.

Example safe message:

"I can provide general dental claims and coding guidance, but I cannot access personal claim records or insurance accounts. For exact claim status, please contact your dental office or insurance carrier."

9. Professional UI Goal

The chatbot UI should be modern, clean, premium, and client-attractive.

It should not look like a simple basic chat box. It should feel like a polished AI assistant built for a professional dental business.

The design should include:

- Clean floating chat launcher
 - Premium card-based chat widget
 - Smooth open/close animation
 - Clear header with assistant identity
 - Professional dental support branding
 - Friendly welcome message
 - Suggested question chips
 - Clean user and assistant message bubbles
 - Loading/typing animation
 - Source display for answers
 - Disclaimer banner
 - Human support handoff option
 - Mobile responsive full-screen view
 - Polished spacing, typography, and visual hierarchy
-

10. Suggested Chatbot Name

Possible chatbot names:

- Dental Support Assistant
- Dental Claims Assistant

- Dental Care Support Bot
- Smile Support Assistant
- Claims Guidance Assistant
- Dental Help Assistant

Recommended name:

Dental Support Assistant

Subtitle:

General dental claims and insurance guidance

11. UI Components Required

11.1 Chat Launcher

A floating button at the bottom-right of the website.

Purpose:

- Open the chatbot
- Create a professional first impression
- Show a small message such as “Need help with dental claims?”

Component:

`ChatLauncher.tsx`

11.2 Chat Widget

The main chatbot container.

Purpose:

- Hold the complete conversation
- Display messages
- Show disclaimer
- Provide user input
- Show suggested questions

Component:

`ChatWidget.tsx`

11.3 Chat Header

The top section of the chatbot.

Should include:

- Bot name
- Small status indicator
- Subtitle
- Close/minimize button

Component:

```
ChatHeader.tsx
```

Example:

Dental Support Assistant

General dental claims guidance

11.4 Disclaimer Banner

A small notice inside the chatbot.

Purpose:

- Make the chatbot safe for public users
- Clearly explain that it does not access private records

Component:

```
DisclaimerBanner.tsx
```

Example:

"This assistant provides general dental claims guidance only. It cannot check personal claim status or access private insurance records."

11.5 Suggested Questions

Clickable question chips to guide users.

Component:

`SuggestedQuestions.tsx`

Suggested questions:

- Why was my dental claim denied?
 - What is a dental procedure code?
 - What does pre-authorization mean?
 - What is the difference between rejected and denied?
 - Why is tooth number required?
 - Why is tooth surface required?
 - What documents may be needed for a dental claim?
 - Why did insurance not pay the full amount?
-

11.6 Message List

Displays all chat messages.

Component:

`MessageList.tsx`

Should support:

- User messages
 - Assistant messages
 - Loading state
 - Error state
 - Empty state
 - Auto-scroll to latest message
-

11.7 Message Bubble

Displays each individual message.

Component:

`MessageBubble.tsx`

Assistant messages should support:

- Paragraphs
- Bullet points
- Short explanations
- Source citations

- Handoff suggestion
-

11.8 Chat Input

Input area where users type questions.

Component:

`ChatInput.tsx`

Should include:

- Textarea
 - Send button
 - Enter to send
 - Shift + Enter for new line
 - Disabled state while loading
 - Empty message validation
-

11.9 Source Citations

Shows which knowledge-base document was used.

Component:

`SourceCitations.tsx`

Example:

Sources:

- Common Claim Failure Reasons
 - Dental Claims FAQ
 - Dental Insurance Glossary
-

11.10 Human Handoff

Shown when the chatbot cannot answer or when the user needs exact claim support.

Component:

`HumanHandoff.tsx`

Example:

"For exact claim details, please contact your dental office or insurance carrier."

Button:

Contact Support

12. Technical Goal

Build a scalable RAG-based chatbot system using:

- Next.js frontend
 - React components
 - TypeScript
 - Tailwind CSS
 - shadcn/ui
 - FastAPI backend
 - Qdrant vector database
 - OpenAI LLM
 - OpenAI embeddings
 - Document upload and chunking system
-

13. Recommended Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Lucide React icons

Backend

- Python
- FastAPI
- Uvicorn
- Python-dotenv

AI Layer

- OpenAI API

- OpenAI embeddings

Vector Database

- Qdrant

Document Processing

- PyPDF
- python-docx
- Text/Markdown parser
- Chunking utility

Deployment

Recommended:

- Frontend: Vercel
- Backend: Render, DigitalOcean, Hetzner, or AWS
- Qdrant: Qdrant Cloud or self-hosted Qdrant

14. System Architecture

User opens website chatbot.

User asks a question.

Frontend sends the question to backend API.

Backend performs a safety check.

Backend converts the question into an embedding.

Backend searches Qdrant for relevant public dental knowledge.

Qdrant returns matching document chunks.

Backend sends retrieved context and user question to OpenAI.

OpenAI generates a professional answer.

Backend returns answer and sources to frontend.

Frontend displays the answer in a polished chatbot UI.

15. Main Backend APIs

15.1 Health Check API

Endpoint:

```
GET /health
```

Purpose:

Check if backend is running.

15.2 Chat API

Endpoint:

```
POST /chat
```

Purpose:

Receive user question and return chatbot answer.

Request example:

```
{
  "message": "Why was my dental claim denied?"
}
```

Response example:

```
{
  "answer": "A dental claim may be denied for several general reasons, such as missing information, service limitations, inactive coverage, or missing required documentation.",
  "sources": [
    {
      "title": "Common Claim Failure Reasons",
      "category": "claim_failure"
    }
  ],
  "handoff_required": false
}
```

15.3 Document Upload API

Endpoint:

```
POST /admin/upload-document
```

Purpose:

Upload approved public knowledge-base documents.

Supported files:

- PDF
- DOCX
- TXT
- Markdown

15.4 Document List API

Endpoint:

```
GET /admin/documents
```

Purpose:

Show uploaded documents in admin dashboard.

15.5 Delete Document API

Endpoint:

```
DELETE /admin/document/{id}
```

Purpose:

Remove a document and its related Qdrant chunks.

15.6 Reindex API

Endpoint:

POST /admin/reindex

Purpose:

Reprocess documents and update Qdrant vector data.

16. Qdrant Collection Design

Collection name:

public_dental_support_knowledge

Each chunk should include metadata:

```
{
  "access_level": "public",
  "category": "claim_failure",
  "topic": "missing_attachment",
  "source_file": "common_claim_failure_reasons.docx",
  "language": "en",
  "version": "v1"
}
```

Recommended categories:

- claims_faq
 - dental_code_guide
 - claim_failure
 - claim_denial
 - insurance_terms
 - pre_authorization
 - support_handoff
 - required_documents
 - appeal_process
-

17. Knowledge Base Documents Required

The first version should use these public-safe documents:

17.1 Dental Claims FAQ

Covers:

- What is a dental claim?
 - How claim processing works
 - How long processing may take
 - What information is usually required
 - What happens after submission
 - When to contact the dental office
 - When to contact the insurance carrier
-

17.2 Dental Code Guide

Covers:

- What dental procedure codes are
 - Why dental codes are used
 - Who selects dental codes
 - Why incorrect codes may cause issues
 - Why tooth number may be required
 - Why tooth surface may be required
 - Why attachments may be needed
-

17.3 Common Claim Failure Reasons

Covers:

- Missing patient information
 - Incorrect subscriber information
 - Inactive insurance coverage
 - Missing provider information
 - Invalid or outdated procedure code
 - Missing tooth number
 - Missing tooth surface
 - Missing attachment
 - Duplicate claim
 - Prior authorization required
 - Service not covered
 - Annual maximum reached
 - Waiting period
 - Frequency limitation
 - Coordination of benefits issue
-

17.4 Rejected vs Denied Claim Guide

Covers:

- What is a rejected claim?
 - What is a denied claim?
 - Difference between rejection and denial
 - What to do after rejection
 - What to do after denial
 - When resubmission may be needed
 - When appeal may be needed
-

17.5 Dental Insurance Glossary

Covers:

- Deductible
 - Copay
 - Coinsurance
 - Annual maximum
 - Waiting period
 - Pre-authorization
 - Coordination of benefits
 - In-network provider
 - Out-of-network provider
 - Covered service
 - Non-covered service
 - Explanation of Benefits
-

17.6 Support Handoff Guide

Covers:

- When chatbot should recommend contacting support
 - What information users should prepare
 - When to contact dental office
 - When to contact insurance carrier
 - When to contact billing team
 - Emergency disclaimer
 - Claim-specific limitation message
-

18. Development Phases

Phase 1: UI Foundation

Goal:

Build a premium, professional chatbot interface with dummy responses.

Tasks:

- Create Next.js project
- Install Tailwind CSS
- Install shadcn/ui
- Build chatbot launcher
- Build chatbot widget
- Build message UI
- Build input component
- Add suggested questions
- Add disclaimer banner
- Add source display placeholder
- Add mobile responsive layout

Success Criteria:

- Chatbot opens and closes smoothly
 - User can type a question
 - Dummy assistant response appears
 - UI looks polished and professional
 - Mobile view works properly
-

Phase 2: Backend API Foundation

Goal:

Create FastAPI backend and connect it with frontend.

Tasks:

- Create FastAPI project
- Add `/health` endpoint
- Add `/chat` endpoint
- Return dummy chatbot response
- Connect frontend chat input with backend
- Add error handling
- Add loading state

Success Criteria:

- Frontend sends message to backend
 - Backend returns response
 - Frontend displays response
 - Error state works if backend is down
-

Phase 3: Qdrant Setup

Goal:

Set up Qdrant vector database.

Tasks:

- Add Docker Compose for Qdrant
- Run Qdrant locally
- Open Qdrant dashboard
- Connect FastAPI with Qdrant
- Create Qdrant collection
- Test Qdrant connection endpoint

Success Criteria:

- Qdrant runs on localhost
 - Backend connects to Qdrant
 - Collection is created successfully
-

Phase 4: Document Upload and Chunking

Goal:

Allow admin to upload public dental knowledge-base documents and store chunks in Qdrant.

Tasks:

- Create document upload API
- Extract text from PDF/DOCX/TXT
- Clean extracted text
- Split text into chunks
- Generate embeddings
- Store chunks in Qdrant
- Save metadata with each chunk

Success Criteria:

- Admin can upload document
 - Text is extracted correctly
 - Chunks are created
 - Chunks are embedded
 - Chunks are stored in Qdrant
 - Source metadata is saved
-

Phase 5: RAG Chat Response

Goal:

Replace dummy response with real RAG answer.

Tasks:

- Convert user question into embedding
- Search relevant chunks in Qdrant
- Retrieve top matching chunks
- Build prompt with retrieved context
- Generate answer using OpenAI
- Return answer with source citations
- Add fallback when answer is not found

Success Criteria:

- User asks dental support question
 - Backend retrieves relevant content
 - Chatbot answers from knowledge base
 - Sources are shown
 - Bot says when information is not available
-

Phase 6: Public Safety and Scope Guardrails

Goal:

Prevent the chatbot from handling private or unsafe requests.

Tasks:

- Detect private claim-status requests
- Detect requests for patient data
- Detect requests containing subscriber ID, SSN, DOB, or private insurance data

- Refuse private data processing
- Suggest contacting dental office or insurance carrier
- Add safe disclaimer in prompt

Success Criteria:

- Bot does not ask for private data
 - Bot does not claim to access real claim status
 - Bot redirects users safely
 - Bot keeps responses general and professional
-

Phase 7: Admin Dashboard

Goal:

Create an admin panel for managing chatbot knowledge.

Tasks:

- Admin document upload page
- Document list
- Delete document option
- Reindex option
- Test question panel
- Prompt settings
- Chat logs view

Success Criteria:

- Admin can manage documents
 - Admin can test chatbot answers
 - Admin can see uploaded document status
 - Admin can improve content over time
-

Phase 8: Final Polish and Deployment

Goal:

Make the chatbot client-ready and production-ready.

Tasks:

- Polish UI animations
- Improve mobile layout
- Add branded colors

- Add professional empty states
- Add error handling
- Add loading skeletons
- Add deployment configs
- Deploy frontend
- Deploy backend
- Deploy/connect Qdrant
- Test production environment

Success Criteria:

- Website chatbot works live
- Backend works live
- Qdrant search works live
- Answers are professional
- UI looks premium
- Client can review demo confidently

19. UI Quality Requirements

The UI should follow these standards:

- Professional dental/healthcare look
- Clean spacing
- Soft rounded corners
- Smooth shadows
- Clear typography
- Calm and trustworthy colors
- Mobile-first responsiveness
- Fast loading
- Clear assistant identity
- Easy-to-click suggested questions
- Clear support handoff
- No clutter
- No confusing technical text for users

Recommended visual style:

- White or off-white background
 - Soft blue, teal, or dental-care accent color
 - Rounded chat card
 - Subtle shadow
 - Clean icons
 - Minimal but premium layout
-

20. Example Welcome Message

"Hi, I'm your Dental Support Assistant. I can help explain general dental claims, coding, insurance terms, claim denials, and common processing issues. I can't access personal claim records, but I can guide you on what information may be needed and what steps to take next."

21. Example Suggested Questions

- Why was my dental claim denied?
 - What is the difference between a rejected and denied claim?
 - What does pre-authorization mean?
 - Why is a tooth number required?
 - Why is a tooth surface required?
 - What documents may be needed for a dental claim?
 - Why did my insurance not pay the full amount?
 - What should I do if my claim needs an attachment?
-

22. Example Safe Refusal

User:

"Can you check my claim status? My claim ID is 12345."

Chatbot:

"I'm sorry, but I can't access or check personal claim records. For exact claim status, please contact your dental office, billing team, or insurance carrier. I can still explain general reasons why a dental claim may be delayed, rejected, or denied."

23. Example RAG-Based Answer

User:

"Why was my dental claim denied?"

Chatbot:

"A dental claim may be denied for several general reasons. Common reasons include inactive coverage, missing required documentation, service limitations, waiting periods, frequency limits, or the service not being covered under the plan. You should review the Explanation of Benefits if available and contact your dental office or insurance carrier for the exact reason."

Sources:

- Common Claim Failure Reasons
 - Dental Insurance Glossary
-

24. Project Success Criteria

The project will be considered successful when:

- The chatbot has a professional and premium UI
 - Public users can ask dental support questions
 - Chatbot answers from approved knowledge-base documents
 - Chatbot shows source references
 - Chatbot does not ask for private information
 - Chatbot safely redirects claim-specific requests
 - Admin can upload and manage documents
 - Qdrant stores and retrieves relevant chunks
 - Responses are clear, helpful, and client-ready
 - The system can be deployed for public website use
-

25. First Milestone

The first milestone is:

Build a professional chatbot UI with dummy API response.

Deliverables:

- Next.js project setup
- Chat launcher
- Chat widget
- Suggested questions
- Message bubbles
- Input field
- Disclaimer banner
- Dummy response from backend

Once this is complete, the project will visually look client-ready even before the RAG knowledge base is connected.

26. Second Milestone

The second milestone is:

Connect chatbot UI with FastAPI backend.

Deliverables:

- FastAPI backend setup
 - `/health` endpoint
 - `/chat` endpoint
 - Frontend API connection
 - Loading state
 - Error state
 - Dummy response from backend
-

27. Third Milestone

The third milestone is:

Connect Qdrant and prepare document indexing.

Deliverables:

- Qdrant running locally
 - Qdrant dashboard working
 - FastAPI connected to Qdrant
 - Qdrant collection created
 - Ready for document upload and chunking
-

28. Fourth Milestone

The fourth milestone is:

Upload dental documents and convert chatbot into real RAG chatbot.

Deliverables:

- Document upload API
- Text extraction
- Chunking
- Embedding generation
- Qdrant storage

- RAG answer generation
 - Source citations
-

29. Final Recommendation

Start development in this order:

1. UI with dummy response
2. Backend API with dummy response
3. Connect UI and API
4. Setup Qdrant
5. Add document upload
6. Add chunking and embeddings
7. Store chunks in Qdrant
8. Add real RAG response
9. Add admin dashboard
10. Polish and deploy

This order is best because the client can see a professional visual demo early, while the technical RAG system is built step by step behind the scenes.